Ders 23 Genişletilmiş Notu: Tekrar Dersi: 3. Ödev ve Vize 1. Sorunun Değerlendirmesi

Ders 23 Genişletilmiş Notu: Tekrar Dersi: 3. Ödev ve Vize 1. Sorunun Değerlendirmesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Büyük Programlarda Analiz
- 2. 3. Ödev: Kelimeleri Alfabetik Sıralamak
- 3. Harf Başlangıç Dizisi ve Arama
- 4. C'de Karakter Dizisi Ataması ve string.h
- 5. Ödevin C Karşılığı
- 6. Vize 1. Soru: Onaltılık Karakter Dizisinin Onluk Karşılığı
- 7. Vize 2. Soru Üzerine: Sütun Silme ve Son Sütundaki Sıfırlar
- 8. Kütüphaneyi Eklemeden strcmp Kullanmak
- 9. 1. Soruya Dönüş: Rakamları Dizide Aramak

Bu ders tekrar dersidir, yeni konu anlatılmadı. Hoca dersi üç işe ayırdı: 3. ödevi konuşmak, birinci vizenin klasik bölümündeki ilk soruyu akış diyagramı üzerinden çözmek ve öğrencilerin sorularını cevaplamak.

1. Büyük Programlarda Analiz

Dersin ilk bölümü, öğrencilerin 3. ödevin analizinde yaşadığı zorluklarla başladı. Küçük problemlerde analiz, her değişkenin her adımda aldığı değeri göstererek yapılıyordu. Ödevdeki program büyüdükçe bu yol işlemez hale geldi.

Soru: Analizi değişken değişken yapamadım. Değerleri programa otomatik yazdırmayı denedim, bir çalıştırma için 300-600 satırlık çıktı oluştu. Bunun yerine yalnız önemli değişkenlerin değiştiği yerlerde çıktı aldım. Analizde hangi kısımları göstermem gerektiğine karar veremiyorum.

Cevap: Küçük problemler için öğrenilen analiz büyük problemlerde de kullanılır, ama bir yerden sonra bu şekilde takip etmek zorlaşır. Bu noktada önemli olan, programın farklı durumlar için verilen girdilere karşılık istenen çıktıları üretip üretmediğini göstermektir. Programa bir girdi verilir ve o durum için beklenen çıktının üretildiği görülür. Analiz buna göre yapılır.

Birçok IDE'nin hata ayıklama (debug) özelliği de analiz için kullanılabilir.

> **Hocanın vurgusu:** Bu kadar uzun bir programda her adımı kağıt üzerinde tek tek göstermeniz beklenmiyor. Farklı durumlar için girilen girdilere karşılık istenen çıktının üretildiğini göstermek önemlidir.

Soru: Programı büyük harflere göre yazdım. Küçük harf girildiğinde hata oluyor. Bu yüzden programın başına büyük harf girilmesini isteyen bir uyarı ekledim ve bulduğum hataları analize yazdım. Bu sorun olur mu?

Cevap: Hayır. Büyük ve küçük harflerin ikisini de kontrol etmek güzel olurdu, ama zorunlu değildi. Sözlük küçük ya da büyük harf üzerinden oluşturulabilir.

2. 3. Ödev: Kelimeleri Alfabetik Sıralamak

Ödev bir sözlük uygulamasıdır. Karışık sırada bir kelime grubu verilir. Önce bu grup alfabetik olarak sıralanır. Sonra daha akıllı bir arama yapabilmek için her harfin başlangıç noktası saklanır.

Hoca şu soruyu sordu: Alfabetik sıralarken normal sıralama algoritmasından farklı olarak neye dikkat etmek gerekir? Sayı sıralamasında iki değer tek karşılaştırmayla kıyaslanır. Kelimelerde ise birden fazla harf vardır, bu yüzden karşılaştırma harf harf yapılır.

2.1 Öğrencilerin Yaklaşımları

Öğrenciler kullandıkları üç yolu anlattı.

- **Gruplayarak sıralamak:** Kelimeler önce birinci harfe göre sıralanır. Sonra her grup kendi içinde ikinci harfe göre sıralanır ve her grupta tek eleman kalana kadar devam edilir. Hoca bunun bir yaklaşım olduğunu, ama çok verimli olduğunu söyleyemeyeceğini belirtti.
- **İç içe iki döngü:** Öğrenci bunu bubble sort'un iyileştirilmiş biçimi olarak tarif etti. Alfabetik sırada en önce gelecek kelime bulunur. Bunun için iki satır harf harf karşılaştırılır. Harfler eşitse sonraki harfe geçilir, eşit değilse küçük olan yeni minimum olur.
- **Uzunluk bularak karşılaştırmak:** Önce bir `while` ile iki kelimenin uzunluğu (`len1`, `len2`) bulunur. Sonra aynı olan harfler atlanır ve ilk farklı harfe göre karar verilir. Hoca bunu şöyle özetledi: harfler aynı olduğu sürece bir şey yapılmaz, ilk farklılıkta hangisi küçükse minimum odur.

  Ek bilgi (derste anlatılmadı): İkinci öğrencinin tarif ettiği, her turda en küçük elemanı bulma fikri büyük olasılıkla bubble sort'tan çok seçmeli sıralamaya (selection sort) benzer.

Üç yaklaşımın ortak noktası hocanın vurguladığı karşılaştırma kuralıdır: harfler eşit olduğu sürece karşılaştırmaya devam edilir. Sonunda biri diğerinden küçük çıkar ve küçük olan yeni minimum olarak belirlenir.

> **Ek bilgi (derste anlatılmadı):** Biri diğerinin başı olan iki kelimede farklı harf bulunmayabilir. C'de karakter dizisinin sonundaki null karakteri (`'\0'`) büyük olasılıkla bütün harflerden küçük olduğu için kısa kelime önce gelir. Aşağıdaki `karsilastir` fonksiyonu bu durumu ayrıca kontrol etmeden doğru sonucu verir.

2.2 Kelimeler Yerine Sıra Dizisini Değiştirmek

Bir öğrenci, algoritmadan bağımsız ve herkesin ekleyebileceği bir fikir önerdi. Sıralama sırasında kelimelerin yerini değiştirmek yerine, kelimelerin sırasını tutan ayrı bir sayı dizisi kullanılır. Yer değiştirme gerektiğinde her karakteri taşımak yerine bu dizideki değerler değiştirilir. İşlemler bitince kelimeler bu eşleştirmeye (mapping) göre yerlerine konur.

Hoca bunun gayet mantıklı bir çözüm olduğunu ve işi daha az karmaşık hale getirdiğini söyledi, sonra fikri bir kez daha anlattı:

- Her kelimenin sırası ayrı bir dizide tutulur.
- İki kelime karşılaştırılır. Küçük olanla büyük olanın yeri değişecekse yalnız bu dizideki pozisyonlar değiştirilir.
- Sıralama bitince dizinin son hali, kelimelerin bulunması gereken pozisyonları gösterir.
- Son adımda kelimeler bu diziye göre yerleştirilir.

Böylece değişiklik her turda kelimelerin kendisi üzerinde yapılmamış olur.

3. Harf Başlangıç Dizisi ve Arama

Sıralamadan sonra yapılması gereken bir iş daha vardır: her yeni harfle başlayan kelime grubunun kaçıncı satırdan başladığını saklayan bir dizi oluşturmak. Hocanın hatırlattığı örnek şöyledir:

Harf	Başlangıç satırı
A, B, C, D, E	0 (bu harflerle başlayan kelime yok)
F	1
G, H, …	0 (L'ye kadar kelime yok)
L	2
S	3
T	5

Bu tablodan S ile başlayan iki kelime olduğu görülür, çünkü S 3. satırdan, sonraki grup T ise 5. satırdan başlar (ara adım eklendi).

Soru: Bu ek dizinin amacı nedir?

Cevap (bir öğrenci): Kelime ararken daha az işlem yapmak. Yer biliniyorsa aramaya doğrudan oradan başlanabilir.

Hoca cevabı tamamladı. Önce aranan kelimenin ilk harfine bakılır. Sonra dizide o harfin gözüne gidilir. O gözde kaç yazıyorsa arama o satırdan başlar. Her arama adımı sayacı bir artırır.

S ile başlayan bir kelime arandığını düşünelim:

- **1. arama:** S'nin ilk kelimesine bakılır. Eşleşmezse birinci arama tamamlanmıştır.
- **2. arama:** İkinci kelimeye bakılır. Eşleşirse kelimenin ikinci aramada bulunduğu söylenir. Eşleşmezse ikinci arama da tamamlanmıştır.
- **3. arama:** Üçüncü bir kelime yoktur. Böylece üç arama sonunda kelimenin olmadığına karar verilir.

4. C'de Karakter Dizisi Ataması ve string.h

Soru: Yer değiştirirken `TMP` diye bir değişken oluşturuyorduk. Bir karakter dizisini ona doğrudan atayamıyor muyuz?

Cevap: Atayamazsınız. Harf harf yazmanız gerekir. Bir de `string.h` kütüphanesi vardır. Karşılaştırma, atama, birleştirme (concatenate) gibi karakter dizisi işlemleri bu kütüphanedeki fonksiyonlarla yapılabilir.

Hoca ödevde bu kütüphanenin kullanılmasını istemedi. Amacı, bu işlerin temelde nasıl yapıldığını ve bu işi yapan fonksiyonların nasıl çalıştığını öğrencilerin görmesiydi.

5. Ödevin C Karşılığı

Hoca derste kod yazmadı. Aşağıdaki kod, 2. ve 3. bölümde anlatılan adımların C karşılığıdır: harf harf karşılaştırma, `string.h` olmadan kopyalama, sıra dizisiyle sıralama, kelimeleri son adımda bir kez yerleştirme, harf başlangıç dizisi ve arama adımlarını sayma. Kod büyük harfli sözlük varsayar. Satır numaraları hocanın örneğindeki gibi 1'den başlar, 0 değeri o harfle başlayan kelime olmadığını gösterir. C'de dizi 0'dan başladığı için arama `bas[h] - 1` numaralı gözden başlar.

```c
#include <stdio.h>
#define MAXK 100   /* en fazla kelime sayisi */
#define MAXH 30    /* bir kelimenin en fazla harf sayisi */

/* a, b'den once geliyorsa negatif, sonra geliyorsa pozitif, esitse 0 */
int karsilastir(char a[], char b[])
{
    int i = 0;
    while (a[i] == b[i] && a[i] != '\0') {
        i++;
    }
    return a[i] - b[i];
}
```

```c
/* string.h olmadan harf harf kopyalama */
void kopyala(char hedef[], char kaynak[])
{
    int i = 0;
    while (kaynak[i] != '\0') {
        hedef[i] = kaynak[i];
        i++;
    }
    hedef[i] = '\0';
}

int main(void)
{
    char kelime[MAXK][MAXH], gecici[MAXK][MAXH], aranan[MAXH];
    int sira[MAXK], bas[26];
    int n, i, j, t, h, adim, bulundu, devam;

    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%s", kelime[i]);
        sira[i] = i;
    }

    /* Siralama: kelimeler degil, sira dizisindeki indisler yer degistirir */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (karsilastir(kelime[sira[j]], kelime[sira[j + 1]]) > 0) {
                t = sira[j];
                sira[j] = sira[j + 1];
                sira[j + 1] = t;
            }
        }
    }

    /* Son adim: kelimeler sira dizisine gore bir kez yerlestirilir */
    for (i = 0; i < n; i++) {
        kopyala(gecici[i], kelime[sira[i]]);
    }
    for (i = 0; i < n; i++) {
        kopyala(kelime[i], gecici[i]);
    }

    /* Harf baslangic dizisi: satirlar 1'den baslar, 0 = bu harfle kelime yok */
    for (h = 0; h < 26; h++) {
        bas[h] = 0;
    }
    for (i = n - 1; i >= 0; i--) {
        bas[kelime[i][0] - 'A'] = i + 1;
    }

    for (i = 0; i < n; i++) {
        printf("%d %s\n", i + 1, kelime[i]);
    }
    for (h = 0; h < 26; h++) {
        if (bas[h] != 0) {
            printf("%c:%d ", 'A' + h, bas[h]);
        }
    }
    printf("\n");
```

```
    /* Arama: ilk harfin baslangic satirindan baslanir */
    while (scanf("%s", aranan) == 1) {
        h = aranan[0] - 'A';
        adim = 0;
        bulundu = 0;
        if (bas[h] != 0) {
            i = bas[h] - 1;
            devam = 1;
            while (devam == 1) {
                adim++;
                if (i >= n || kelime[i][0] != aranan[0]) {
                    devam = 0;      /* bu harfle baslayan kelime kalmadi */
                } else if (karsilastir(kelime[i], aranan) == 0) {
                    bulundu = 1;
                    devam = 0;
                } else {
                    i++;
                }
            }
        }
        if (bulundu == 1) {
            printf("%s: %d. aramada bulundu\n", aranan, adim);
        } else {
            printf("%s: yok (%d arama)\n", aranan, adim);
        }
    }
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlendi. Hocanın örneğindeki düzene uyan beş kelimelik bir listeyle çalıştırıldığında başlangıç değerleri F:1, L:2, S:3, T:5 çıktı. S grubundaki ikinci kelime 2. aramada bulundu, S ile başlayıp listede olmayan kelime için 3 arama sayıldı.

6. Vize 1. Soru: Onaltılık Karakter Dizisinin Onluk Karşılığı

Hoca birinci vizenin klasik bölümündeki ilk soruyu ekranda bir akış diyagramıyla gösterdi. Soruda onaltılık bir sayı karakter dizisi olarak verilir ve onluk karşılığı istenir. Hoca kağıtlarda birçok farklı çözüm gördüğünü söyledi. En büyük sorun, karakter olarak verilen bir grupla nasıl işlem yapılacağının bilinmemesiydi.

Örnek sayı `1 0 A F`'dir. Basamakların değerleri sağdan sola 16^0, 16^1, 16^2, 16^3'tür:

$$1 \cdot 16^3 + 0 \cdot 16^2 + 10 \cdot 16^1 + 15 \cdot 16^0$$

6.1 Karakterden Sayıya

Dizinin gözlerindeki değerler sayı değil, karakterdir. Bu yüzden tek tırnakla `'1'`, `'0'`, `'A'`, `'F'` diye düşünülmelidir. Bunların sayısal karşılıkları 49, 48, 65 gibi değerlerdir. Yani `'1'` ile doğrudan işlem yapılırsa 1 ile değil 49 ile işlem yapılmış olur. Gerçek rakamsal değer ancak bir çıkarma işlemiyle elde edilir.

Elimizde iki tip bilgi vardır: `'0'` ile `'9'` arasındakiler ve `'A'` ile `'F'` arasındakiler.

- Karakter `'0'` ile `'9'` arasındaysa `'0'` başlangıç noktası çıkarılır: `tmp = A[I] - '0'`.
- Değilse `'A'` ile `'F'` arasındadır. `'A'` çıkarılır ve 10 eklenir: `tmp = A[I] - 'A' + 10`. Böylece `'A'` geldiğinde 10 değeri elde edilir.

  Ders sonrası not: Hoca bu işlemi "sıfırdan çıkarmak" ve "A'dan çıkarmak" diye anlattı. Doğrusu, `'0'` ya da `'A'` karakterinin eldeki karakterden çıkarılmasıdır. Ekrandaki formül de büyük olasılıkla `A[I] - '0'` biçimindedir.

Bilgilerin doğru verildiği varsayıldığı için iki aralığı da kontrol etmek gerekmez. Biri kontrol edilir, değilse diğeri kabul edilir. Hoca kendi çözümünde iki ayrı kontrol yapmadı.

> **Hocanın vurgusu:** Bu tarz karakter sorularında ASCII tablosundaki değerleri ezberlemek zorunda değilsiniz ve bunları kullanmanızı istemiyorum. 49, 55, 65 gibi sayılar yerine karakter sabitlerini kullanın. Karakter sabitleri ve bu kontroller üzerinde derste çok durduk, örnek yaptık, ödevini de verdik.

Hoca, herkesin bu konuda ayrı bir hesap yaptığını söyledi: Kimi 7 rakamından çıkarmış, çünkü aradaki fark 10 oluyormuş. Kimi de 55 yazmış.

> **Ek bilgi (derste anlatılmadı):** Bu iki hesap büyük olasılıkla aynı şeydir. `'7'` karakterinin ASCII değeri büyük olasılıkla 55, `'A'` karakterininki 65'tir. Bu yüzden `'A' - '7'` ve `'A' - 55` ikisi de 10 verir. Yani çıkarılan 7 sayısı değil, `'7'` karakteridir. Python ile de bu değerler elde edildi.

6.2 Neden Dizinin Sonundan Başlanır?

Hocanın çözümü `N`'den, yani `F`'den başlar. Sebebi, en sağdaki basamağın $16^0 = 1$ ile başlamasıdır. Böylece `X = 1` ve `toplam = 0` ilk değerleri verilir. Her turda `X` 16 ile çarpılarak 16^1, 16^2, 16^3 elde edilir. Üs hesabı için ayrı bir döngü gerekmez.

6.3 Akış Diyagramı

Akış diyagramının adımları şöyledir:

1. `N` ve `A[N]` okunur.
2. `X = 1`, `toplam = 0` atanır.
3. `I ← N || -1 || 1` döngüsü kurulur. Dizi sağdan sola gezilir.
4. `A[I] <= '9' AND A[I] >= '0'` ise `tmp = A[I] - '0'`, değilse `tmp = A[I] - 'A' + 10` olur.
5. `toplam = toplam + tmp * X` ve `X = X * 16` yapılır.
6. Döngü bitince `toplam` yazdırılır.

6.4 Sık Yapılan Hatalar

Hoca sınav kağıtlarında gördüğü hataları tek tek saydı.

- **Çarpanı 16^1 ile başlatmak:** Döngüye bir kez girildiği için, ilk işleme giren son göz 16^0 yerine 16^1 ile çarpılır. En sık yapılan hatalardan biridir.
- **Karakterle sayı gibi işlem yapmak:** `'0'` ya da `'A'` çıkarılmadan doğrudan işlem yapmak. Hocanın gördüğü en büyük sorundur.
- **ASCII değerlerini kullanmak:** 49, 55, 65 gibi sayılar yazmak.
- **Çarpanı her basamak için yeniden hesaplamak:** Tersten başlayıp ayrı bir `for` ile her seferinde 16^0, 16^1, 16^2'yi yeniden hesaplamak. Dizinin birinci gözünden başlayıp 16^3'ü de yine döngüyle hesaplamak ve bir sonraki harfte 16'yı tekrar tekrar kendisiyle çarpmak da aynı durumdur. Bunlar gereksiz işlemdir. Her turda çarpanı 16 ile çarpmak yeterlidir.
- **`pow` fonksiyonu kullanmak:** Üssü göstermek için bu fonksiyona başvurmak da hocanın saydığı sıkıntılar arasındadır.
- **`toplam`'ı sıfırlamamak:** `toplam`'a ilk değer verilmemesi bir hatadır.
- **İlk değerleri döngünün içinde atamak:** `X = 1` ve `toplam = 0` döngünün içine yazılırsa her turda sıfırlanır ve toplam değeri kaybolur.
- **Çok sayıda `if` kullanmak:** A'ya eşit mi, B'ye eşit mi, C'ye eşit mi diye değerleri tek tek kontrol etmek.
- **Dizinin gözlerine değerleri elle atamak:** Gözlere 15, 14, 13, 12 gibi değerler ya da A, F gibi değerler atamak.

6.5 Analiz

Hoca analizin ekrandaki gibi kısaca gösterilmesinin, algoritmanın doğru ilerlediğini göstermek için yeterli olduğunu söyledi. Bununla birlikte analizin tamamlanmış olması beklenir. Ekrandaki tablo her satırda döngüye giriş değerlerini gösterir:

I	X	toplam	A[I]	tmp
4	1	0	F	15
3	16	15	A	10
2	256	175	0	0
1	4096	175	1	1

Her satırdaki toplam şöyle güncellenir (ara adım eklendi):

- $I = 4$: tmp $= 15$, toplam $= 0 + 15 \cdot 1 = 15$, $X = 1 \cdot 16 = 16$.
- $I = 3$: tmp $= 10$, toplam $= 15 + 10 \cdot 16 = 175$, $X = 16 \cdot 16 = 256$.
- $I = 2$: tmp $= 0$, toplam $= 175 + 0 \cdot 256 = 175$, $X = 256 \cdot 16 = 4096$.
- $I = 1$: tmp $= 1$, toplam $= 175 + 1 \cdot 4096 = 4271$.

Sonuç olarak `1 0 A F` sayısının onluk karşılığı 4271'dir.

6.6 C Karşılığı

Hoca ekranda akış diyagramını gösterdi, kod yazmadı. Aşağıdaki kod bu akış diyagramının C karşılığıdır. C'de dizi 0'dan başladığı için indisler bir azaltılmıştır: döngü `N - 1`'den başlar, `0`'da biter.

```c
#include <stdio.h>
#define MAX 100

int main(void)
{
    char A[MAX];
    int N, I, X, toplam, tmp;

    scanf("%d", &N);
    for (I = 0; I < N; I++) {
        scanf(" %c", &A[I]);
    }

    X = 1;
    toplam = 0;
    for (I = N - 1; I >= 0; I--) {
        if (A[I] <= '9' && A[I] >= '0') {
            tmp = A[I] - '0';
        } else {
            tmp = A[I] - 'A' + 10;
        }
        toplam = toplam + tmp * X;
        X = X * 16;
    }
    printf("%d\n", toplam);
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlendi. Girdi olarak `4` ve `1 0 A F` verildiğinde `4271` yazdı.

7. Vize 2. Soru Üzerine: Sütun Silme ve Son Sütundaki Sıfırlar

Hoca, fırsat varken ikinci soruyu da ekranda gösterdi: $N \times M$'lik bir matrisin X numaralı sütununu silen algoritmanın akış diyagramı. Bu derste sorunun çözümü anlatılmadı, yalnız sorular alındı. Konuşmadan anlaşılan iki adım şudur: Sütunlar sola kaydırılır ve son sütunda sıfırlar görünür.

Soru: Sütunu kaydırdığımda son sütun C'de kendiliğinden sıfırlanıyordu. Ekrana boş değil sıfır yazdırıyordu. Bu yüzden akış diyagramına ayrı bir sıfırlama adımı eklemedim. (Ortam: Dev-C++.)

Cevap: Hoca önce sütunun nasıl kendiliğinden sıfırlandığını sordu. Başka bir öğrenci de benzer bir durum yaşadığını, Windows 10 üzerinde Dev-C++'ta dört değişkenden ilk değer verilmeyen ikisinin sıfır görünüp diğer ikisinin görünmediğini söyledi.

Hoca iki durumu ayırdı. Değişkenlerin kendiliğinden bir değer alıyor görünmesi, o anda bellekte bulunan değerin kullanılmasından kaynaklanır. O bellek bölgesi o anda sıfırsa değişken de sıfır görünür. Ama sütun kaydırma örneği farklıdır. Bir sütun diğerinin üzerine kaydırıldığında son sütunda kendi eski değerleri durur. Bu yüzden sıfırların olduğu yerde 4, 8, 12 gibi değerlerin görünmesi beklenir. Hoca sıfırın neden oluştuğunu anlayamadığını ve kodu görerek yorum yapabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Öğrencinin gördüğü sıfırların sebebi büyük olasılıkla kaydırma döngüsünün son sütunu da kapsamasıdır. Bu durumda son sütuna, matrisin kullanılan alanının dışındaki `A[I][M+1]` gözü kopyalanır. Sabit boyutla tanımlanmış büyük bir matriste bu göz atama yapılmamış bir gözdür ve bellekte 0 olabilir. Global tanımlı bir matrisle gcc'de denendiğinde son sütunda 0 görüldü. Bu sonuca güvenilmemeli, sıfırlama adımı açıkça yazılmalıdır.

8. Kütüphaneyi Eklemeden strcmp Kullanmak

Soru: Kütüphaneyi eklemeden o kütüphanedeki bir fonksiyonu kullansak çalışır mı? `string.h`'ı eklemeden `strcmp` yazdım ve kod çalıştı. Ortam Windows 10 üzerinde Dev-C++.

Cevap: Çalışmaması gerekir. `strcmp`, `string.h`'ın bir fonksiyonudur. Eklenmediğinde derleme sırasında hata mesajı verilir. Bu durumun neden oluştuğuna kodu görmeden yorum yapılamaz. Hoca kodu kendi tarafında derleyip bakacağını söyledi.

> **Ders sonrası not:** Hoca, kütüphane eklenmezse normalde derleme hatası alınacağını söyledi; bu derleyiciye ve C standardına göre değişir. Eski standartlarda bildirilmemiş fonksiyon büyük olasılıkla örtük olarak bildirilmiş sayılır. `strcmp` standart kütüphanede bulunduğu için program bağlanıp çalışabilir. gcc 15.2 ile denendiğinde varsayılan ayarlarda `implicit declaration of function 'strcmp'` hatası alındı, `-std=gnu89` ile yalnız uyarı verip çalıştı. Öğrencinin gözlemi büyük olasılıkla bu davranıştan kaynaklanır. Doğrusu her durumda `#include <string.h>` yazmaktır.

9. 1. Soruya Dönüş: Rakamları Dizide Aramak

Dersin sonunda bir öğrenci birinci soruya döndü.

Soru: Onaltılık rakamları önce bir karakter dizisine koydum. Verilen karakteri bu dizide tek tek aradım. Örneğin B verilmişse 12. sırada buldum ve bulduğum sayıya göre işlem yaptım. Bu doğru mu?

Cevap: Bu, verilen karakteri dizideki elemanlarla tek tek karşılaştırmak demektir. Çözüm yanlış değildir, ama istenen çözüm şekli de değildir. 16 değeri tek tek karşılaştırmak iyi bir çözüm sayılmaz.

> **Ders sonrası not:** Öğrenci B'nin 12. sırada olduğunu söyledi. B'nin sayısal değeri 11'dir. 0'dan başlayan numaralamada gözü 11, 1'den başlayan sayımda 12. sıradır. Öğrenci büyük olasılıkla ikinci sayımı kastetmiştir.

Soru: Akış diyagramında kendimiz, örneğin 10 elemanlı, değerleri önceden belli bir dizi belirtmek istersek bunu nasıl gösteririz?

Cevap: Gözlere tek tek atama yapılır: dizinin 0. gözüne 0, 1. gözüne 1 gibi.

Soru: Hepsini toplu olarak gösteremez miyiz?

Cevap: C'deki parantezli ilk değer verme biçimi kastediliyorsa o da gösterilebilir. Bunda bir yanlışlık yoktur, çok büyük bir sorun değildir.

Dersin Toparlaması

Ödev tarafında iki fikir öne çıktı. Birincisi, kelime karşılaştırma kuralıdır: harfler eşit oldukça devam edilir, ilk farklı harfte karar verilir. Kelimeleri taşımak yerine sıra dizisini değiştirip son

adımda bir kez yerleştirmek işi kolaylaştırır. İkincisi, harf başlangıç dizisidir: arama, aranan kelimenin ilk harfinin saklanan satırından başlar. Büyük programların analizinde de her adımı göstermek yerine farklı durumlar için beklenen çıktıların üretildiği gösterilir.

Vize sorusunda ise iki temel beceri öne çıktı: karakterden sayısal değeri karakter sabitleriyle elde etmek (`c - '0'`, `c - 'A' + 10`) ve üssü yeniden hesaplamak yerine çarpanı 1’den başlatıp her turda 16 ile çarpmak. Hoca, kağıtlardaki hataların bazı konuların henüz oturmadığını gösterdiğini söyledi ve anlaşılmayan yerlerin tekrar haftasında sorulmasını istedi. Perşembe günkü derste tekrar amacıyla birkaç örnek soru çözüleceğini belirtti.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.